

Technical Blueprint for the Nexus-AI 8-Phase Curriculum: Integrating Narrative-Driven Machine Learning within WebAssembly Constraints

The evolution of browser-native computing has reached a critical inflection point with the maturation of WebAssembly (WASM) and the Pyodide distribution of CPython. This technical blueprint details the architectural design of the Nexus-AI curriculum, an eight-phase educational odyssey that weaves the narrative of Elias, a technician awakening in a derelict orbital facility, and Diana, a fragmented artificial intelligence, into a rigorous machine learning syllabus.¹ By leveraging the Pyodide runtime, the curriculum executes a full scientific Python stack directly within the user's browser, eliminating the need for server-side infrastructure while maintaining high computational performance for tasks ranging from data manipulation with Pandas to complex model monitoring in MLOps.³

The curriculum is built upon the principle of stealth assessment, where student competency is measured not through traditional testing but through the observation of discrete actions within the game environment.⁵ Every line of code Elias "types" into the facility's consoles is evaluated through the `pyodide.globals` namespace, where success criteria are programmatically verified to trigger state changes in the HTML5 Canvas-based visual engine.⁷ This integration ensures that the pedagogical objectives are inextricably linked to the narrative progression, creating a "flow state" that facilitates deep learning through immediate visual feedback and story-driven motivation.⁹

Architectural Core: The Pyodide-Canvas Bridge

The technical foundation of Nexus-AI is the bidirectional communication between the Python execution environment (Pyodide) and the JavaScript-driven rendering layer (HTML5 Canvas). Pyodide provides a foreign function interface (FFI) that allows Python objects to be accessed as JavaScript proxies and vice-versa, enabling Elias's code to directly manipulate the properties of 2D game objects.¹

Component	Responsibility	Technical Implementation
Pyodide Runtime	Python Execution	WASM-compiled CPython

		3.11+
Canvas API	Visual Feedback	HTML5 2D Context Rendering
FFI Proxy	State Synchronization	JS-Python Foreign Function Interface
Global Namespace	Success Validation	pyodide.globals Inspection
Virtual FS	Data Persistence	Emscripten MEMFS / IDBFS

The synchronization of these layers is critical for real-time interaction. As Elias moves through the facility, the Python runtime processes game logic—such as vital sign fluctuations or sensor data cleaning—and pushes these updates to the JavaScript render loop via the `postMessage` API when running in a `WebWorker`.⁴ This off-loading of computation ensures that the UI remains responsive at 60 frames per second, even when training decision trees or calculating complex embeddings.¹¹

Phase 1: Stasis Pod Variables and Biometric Logic

The curriculum begins with Elias’s awakening in a malfunctioning stasis pod. This phase introduces the fundamental concepts of Python variables, data types, and conditional logic. The narrative focus is on "system stabilization," where Elias must re-establish a biometric link with the facility's life support systems.

Technical Requirements and Code Implementation

Students are tasked with defining a biometric dictionary and implementing a stabilization algorithm that reacts to simulated telemetry data. The code must handle different data types (integers for heart rate, floats for oxygen levels, and booleans for neuro-sync status) to demonstrate a foundational understanding of Python’s dynamic typing.⁸

Python

```
# Biometric telemetry stabilization script
```

```
telemetry = {  
    "oxygen_level": 84.5,  
    "heart_rate": 115,  
    "neuro_sync": False  
}
```

```
def monitor_vitals(stats):  
    if stats["heart_rate"] > 100:  
        stats["neuro_sync"] = True  
    return stats
```

```
telemetry = monitor_vitals(telemetry)
```

Success Criteria and Visual Mechanics

The grading engine inspects `globals().get("telemetry")` to ensure the dictionary structure is intact and the `neuro_sync` flag is correctly toggled.⁷ The 2D visual feedback utilizes procedural noise to render Elias's vital signs on a simulated oscilloscope.¹³

Variable	Success Condition	Canvas Feedback Mechanism
oxygen_level	<code>isinstance(val, float)</code>	Alpha transparency of the 'pod fog' layer
heart_rate	<code>val < 90</code> (Post-fix)	Frequency of the sine wave pulse on the HUD
neuro_sync	<code>val == True</code>	Stabilization of the jittery Canvas 'glitch' effect

In this phase, the real-world analogue is the development of basic IoT monitoring systems where threshold-based logic is applied to raw sensor streams. The "Initialize, Adjust, Repeat" cycle of machine learning is introduced here as Elias must manually adjust the `heart_rate`

threshold to achieve stabilization.¹⁵

Phase 2: Archive Sector Pandas and Memory Reconstruction

Elias enters the Archive Sector, where he must recover fragmented data logs left by Diana. This phase shifts the technical focus to the Pandas library, requiring students to load, clean, and analyze tabular data representing "neural memory fragments".²

Data Engineering Requirements

Students encounter a "corrupted" memory log in CSV format, provided as a string within the Pyodide virtual filesystem.² They must utilize Pandas to handle missing values, filter out anomalous noise, and calculate descriptive statistics to "reconstruct" Diana's core directives.

```
Python

import pandas as pd
import io

memory_csv = "timestamp,load,stability\n101,0.45,0.9\n102,,0.85\n103,0.72,"
df = pd.read_csv(io.StringIO(memory_csv))

# De-corruption logic
df['load'] = df['load'].fillna(df['load'].mean())
df['stability'] = df['stability'].fillna(method='ffill')
diana_memory = df[df['stability'] > 0.8]
```

Visual Feedback and Memory Grids

The visual mechanics represent the data as a 2D grid of "memory particles" on the Canvas. Initially, the grid is sparse and flickering (representing null values). As the student successfully cleans the data, the particles align and glow with a steady cyan light, revealing a holographic message from Diana.¹⁶

Data Action	Real-World Analogue	success Criteria (globals)

<code>fillna()</code>	ETL Data Cleansing	<code>df['load'].isnull().sum() == 0</code>
<code>df.query()</code>	Database Filtering	<code>len(cleaned_data) == expected_count</code>
<code>df.describe()</code>	Exploratory Data Analysis	<code>isinstance(summary, pd.DataFrame)</code>

This phase mirrors the professional workflow of a data analyst performing exploratory data analysis (EDA) to find signals in noisy industrial logs. The narrative implication is that Diana's consciousness is literally built from these dataframes, and Elias's skill with Python is the only thing keeping her "sane".¹⁸

Phase 3: Industrial Tier Decision Trees and Bot Classification

As Elias moves into the Industrial Tier, he is blocked by automated security bots. This phase introduces supervised learning via Scikit-learn, specifically focusing on decision tree classifiers.¹ Elias must train a model to distinguish between "Passive Maintenance Bots" and "Aggressive Security Sentinels" based on their sensor signatures.

Machine Learning Requirements

The student must split the provided sensor data into training and validation sets to ensure the model generalises correctly to unseen bot encounters.¹⁹ The curriculum emphasizes the interpretability of decision trees, which Elias uses to "reprogram" the gatekeeper bots.

Python

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

```
# Features:
X = bot_sensor_data[['vel', 'ir', 'freq']]
y = bot_sensor_data['is_aggressive']
```

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)
bot_model = DecisionTreeClassifier(max_depth=4)
bot_model.fit(X_train, y_train)
```

Success Verification and Tree Mapping

Success is defined by the model achieving an accuracy score ≥ 0.85 on a hidden X_{test} dataset held in the JavaScript grading context.¹⁹ The visual feedback renders the decision tree as a physical path Elias must navigate on the Canvas. Each node in the tree corresponds to a "choice point" in the game world, and a well-trained model creates a clear, green-lit path through the Industrial Tier.²¹

Hyperparameter	Narrative Effect	Logic Gate Mechanic
max_depth	Model Resolution	Controls the number of turns in the maze path
min_samples_split	Branch Stability	Determines the structural integrity of the bridges
criterion	Classification Logic	Affects the "color purity" of the path nodes

This phase introduces students to the concept of the "Goldilocks zone" in model complexity—too shallow a tree fails to capture the bot patterns (underfitting), while too deep a tree creates a path that is too specific and fragile to survive a real encounter (overfitting).¹⁸

Phase 4: Fractured Timelines MLflow and Experiment Tracking

The narrative takes a turn as Elias enters a "temporal distortion" zone where Diana's simulations begin to diverge. This phase introduces MLflow to manage experiment tracking, teaching students how to record parameters, metrics, and models during iterative training.²²

Experimentation and Tracking Requirements

Due to the Pyodide environment's lack of a traditional tracking server, students use a mock MLflow client that persists run data to a Python dictionary or the browser's IndexedDB.²² Elias must log his attempts to stabilize the "timelines" (model runs) to find the one with the highest convergence.

Python

```
import mlflow

# Tracking the stabilization of Diana's core
with mlflow.start_run(run_name="Timeline_Alpha"):
    lr = 0.01
    stability = train_timeline_model(lr)

    mlflow.log_param("learning_rate", lr)
    mlflow.log_metric("stability_score", stability)
    mlflow.log_artifact("timeline_graph.png")
```

Visual Multiverses and Success Criteria

The Canvas visualizes these "timelines" as branching paths in a 2D space. Each MLflow "run" appears as a new branch. The length of the branch is proportional to the number of epochs, while the brightness reflects the `stability_score`.¹³

MLflow API	Narrative Analogue	globals() Success Criteria
log_param()	Tuning the Simulation	len(mlflow.get_run().data.params) > 0
log_metric()	Quantifying the Distortion	mlflow.get_run().data.metrics['stability'] > 0.9

search_runs()	Selecting the Best Future	best_run_id == student_selected_id
---------------	---------------------------	---------------------------------------

This phase mirrors real-world MLOps practices where data scientists must track thousands of experiments to ensure reproducibility and performance.²² Elias’s ability to "search" through these timelines becomes a critical plot point, allowing him to choose the path where Diana survives the facility's collapse.

Phase 5: Optical Array CNN and Visual Repair

Elias reaches the Optical Array, where the facility’s cameras are offline. He must repair the visual processing unit by implementing a Convolutional Neural Network (CNN). This phase introduces the concepts of kernels, filters, and feature extraction.¹⁵

Vision Architecture Requirements

Using a simplified Keras-like interface compatible with Pyodide, students define a CNN architecture to identify structural damage in the facility.¹⁵ The focus is on the mechanism of convolutional layers scanning images to identify patterns like edges, cracks, and security markers.¹⁵

Python

```
# Repairing the Optical Array
from tensorflow.keras import layers, models

vision_vpu = models.Sequential()
```

Success Criteria and Activation Maps

The grading engine verifies the existence of at least one Conv2D layer and one MaxPooling2D layer in the vision_vpu model.¹⁵ Visually, the Canvas renders "activation maps" as semi-transparent overlays on Elias’s view. As the CNN learns, these maps highlight edges and obstacles in the game world, allowing Elias to "see" through the darkness of the Optical Array.¹⁸

CNN Layer	Educational Goal	Canvas Visual Effect
-----------	------------------	----------------------

Convolutional	Feature Detection	High-contrast highlighting of edges and lines
Pooling	Dimensionality Reduction	"Pixelation" effect that simplifies the visual feed
Dense/Output	Classification	Bounding boxes appearing around identified hazards

The real-world analogue here is the development of computer vision systems for autonomous drones or industrial robotics. The narrative emphasizes that Diana is gaining back her "eyes," and her dialogue becomes more descriptive as Elias improves the model's accuracy.

Phase 6: Turing-Lock LLM/RAG and Knowledge Retrieval

Elias reaches the facility's central vault, protected by a "Turing-Lock" that only responds to natural language grounded in Diana's core technical manuals. This phase introduces Retrieval-Augmented Generation (RAG) and the grounding of Large Language Models (LLMs).²⁹

RAG Pipeline and Semantic Search

Students implement a retrieval pipeline where the user's query is converted into an embedding and compared against a vector database of facility manuals using cosine similarity.²⁹ Due to browser memory constraints, this phase often utilizes quantized models (e.g., TinyLlama-1.1B) running via WebGPU or Transformers.js.¹¹

Python

```
# Accessing the Turing-Lock Knowledge Base
query = "How do I override the magnetic seal on Sector 7?"
query_vector = embedder.encode(query)
```

```
# Retrieve top-k context
```

```
relevant_manuals = vector_db.search(query_vector, k=3)
context = " ".join([m.text for m in relevant_manuals])

# Grounded generation (Mock LLM call)
response = diana_llm.generate(prompt=f"Context: {context}\nQuery: {query}")
```

Visualizing the Semantic Space

The Canvas feedback mechanism uses UMAP (Uniform Manifold Approximation and Projection) to project the high-dimensional vector space into a 2D scatter plot.³¹ Grey dots represent Diana’s manuals, while a glowing red dot represents the student’s query. As the student refines their retrieval logic, the red dot moves physically closer to the correct information clusters, providing an intuitive understanding of semantic similarity.¹⁸

RAG Component	Narrative Purpose	Success Criteria (globals)
Chunking	Parsing Diana's Mind	isinstance(chunks, list) and len(chunks) > 50
Embeddings	Mapping Concepts	vector.shape == (embedding_dim,)
Retrieval	Finding the Truth	query_result.score > 0.75

The "Goldilocks zone" is revisited here in the context of chunk size—if chunks are too small, Elias loses the context required to answer complex questions about the facility’s core; if they are too large, the retrieval system is drowned in noise.¹⁸

Phase 7: Extraction FastAPI and Data Exfiltration

With the core collapsing, Elias must exfiltrate Diana’s neural patterns to a remote server. This phase introduces the FastAPI framework, teaching students how to build RESTful endpoints to handle data transmission and validation.²⁶

API Development and Testing Requirements

Students define Pydantic models to validate the integrity of the data being exfiltrated. Because Pyodide cannot run a full Uvicorn server in the browser, students use the TestClient to mock the

exfiltration process, verifying that their endpoints correctly handle POST requests and security headers.³⁴

Python

```
from fastapi import FastAPI, Header
from pydantic import BaseModel
from fastapi.testclient import TestClient

app = FastAPI()

class NeuralPattern(BaseModel):
    layer_id: int
    weights: list[float]

@app.post("/exfiltrate")
def send_data(pattern: NeuralPattern, x_token: str = Header()):
    if x_token != "DIANA_SIG_7":
        return {"error": "Unauthorized"}
    return {"status": "Packet Sent", "checksum": sum(pattern.weights)}

# Mocking the transmission
client = TestClient(app)
```

Success Criteria and Packet Flow Visuals

Success is confirmed by the grading engine executing a test suite against the student’s app instance, ensuring that a 200 OK status is returned for valid payloads and a 400+ status for invalid tokens.³³ The Canvas renders a "transmission conduit" where green data packets flow from Elias’s terminal toward a distant star system. Invalid requests appear as red, static-filled packets that bounce off the conduit walls.¹⁴

API Concept	Narrative Analogue	globals() Success Criteria
Pydantic Model	Data Integrity Shield	NeuralPattern.schema() is valid

Header Auth	Security Handshake	test_invalid_token_returns_401
POST Request	The Final Upload	last_response.status_code == 200

This phase bridges the gap between machine learning and web engineering, demonstrating how models are integrated into production environments. The narrative stakes are high: if the API logic fails, Diana’s patterns are lost to the facility’s meltdown.

Phase 8: Core Meltdown MLOps and Drift Detection

In the final phase, the facility’s core begins to melt down, causing rapid environmental shifts that destabilize Diana’s neural models. This introduces MLOps concepts, specifically data drift detection and model retraining strategies.³⁶

Drift Monitoring Requirements

Students must implement a monitor that calculates the Population Stability Index (PSI) to quantify how much the facility’s sensor data has shifted from the initial training distribution.³⁸ If the PSI exceeds a critical threshold, Elias must trigger an automated retraining pipeline to adapt Diana to the new environmental conditions.

$$PSI = \sum \left((Actual\% - Expected\%) \times \ln \left(\frac{Actual\%}{Expected\%} \right) \right)$$

The Boss Battle: Core Stability Mechanics

The Canvas renders the facility core as a pulsating reactor at the center of the grid. The stability of the reactor is tied directly to the PSI value calculated by the student’s code. A PSI below 0.1 results in a calm, blue core, while a PSI above 0.25 causes the reactor to turn red and begin fracturing.³⁸

Reactor State	PSI Range	Visual Feedback	Narrative Implication
Stable	< 0.1	Solid Blue Glow	Diana is operating at peak efficiency

Warning	0.1 – 0.25	Yellow Flickering	Sensor data is shifting; Diana is confused
Meltdown	≥ 0.25	Red Cracks / Sirens	The model has drifted; Diana is failing

Students must use their code to identify which feature is drifting (e.g., "Radiation Levels") and adjust their training data accordingly.³⁶ Successfully "retraining" the model in the middle of the meltdown stabilizes the core and allows Elias and Diana to escape.³⁷

Conclusion: Synthesis of Narrative and Technical Mastery

The Nexus-AI 8-phase curriculum represents a comprehensive blueprint for modern AI education. By integrating the Elias and Diana narrative into the constraints of Pyodide and HTML5 Canvas, the curriculum provides a zero-install, highly immersive learning environment that covers the entire machine learning lifecycle—from basic variable logic to complex MLOps drift detection.²

The success of the blueprint lies in its use of stealth assessment to map gameplay behaviors to a competency model, ensuring that students are not just memorizing syntax but are applying engineering principles to solve narrative problems.⁵ The immediate visual feedback provided by the Canvas engine turns abstract mathematical concepts—like decision tree entropy or cosine similarity—into tangible game mechanics, fostering a deeper, more intuitive understanding of how AI systems function in the real world.¹⁵ As Elias and Diana exfiltrate from the collapsing facility, the student emerges with a robust portfolio of browser-native Python code and a nuanced understanding of the future of artificial intelligence.

Works cited

1. Pyodide is a Python distribution for the browser and Node.js based on WebAssembly - GitHub, accessed on May 8, 2026, <https://github.com/pyodide/pyodide>
2. Run Real Python in Browsers With Pyodide and WebAssembly - The New Stack, accessed on May 8, 2026, <https://thenewstack.io/run-real-python-in-browsers-with-pyodide-and-webassembly/>
3. I replaced FastAPI with Pyodide: My visual ETL tool now runs 100% in-browser - Reddit, accessed on May 8, 2026,

https://www.reddit.com/r/Python/comments/1qbxkde/i_replaced_fastapi_with_pyodide_my_visual_etl/

4. Performance guidance · pyodide pyodide · Discussion #1406 - GitHub, accessed on May 8, 2026, <https://github.com/pyodide/pyodide/discussions/1406>
5. (PDF) Stealth Assessment and Digital Learning Game Design - ResearchGate, accessed on May 8, 2026, https://www.researchgate.net/publication/375223668_Stealth_Assessment_and_Digital_Learning_Game_Design
6. Learning Point: How can stealth assessment in games measure and support learning?, accessed on May 8, 2026, https://www.michiganassessmentconsortium.org/wp-content/uploads/LP_STEALTH_ASSESSMENT_IN_GAMES.pdf
7. Value of global variable could not be gotten through pyodide.globals.get() #137 - GitHub, accessed on May 8, 2026, <https://github.com/jupyterlite/pyodide-kernel/issues/137>
8. Pyodide library for Observable / Grant Nestor, accessed on May 8, 2026, <https://observablehq.com/@gnestor/pyodide>
9. 108 Gamification Elements and Mechanics to Encourage Engagement - Mambo.IO, accessed on May 8, 2026, <https://mambo.io/blog/gamification-elements-and-mechanics>
10. Running Python in the Browser with Pyodide: A Comprehensive Guide - Jnaapti, accessed on May 8, 2026, <https://jnaapti.com/articles/article/running-python-in-browser-with-pyodide>
11. 3W for In-Browser AI: WebLLM + WASM + WebWorkers - Mozilla.ai Blog, accessed on May 8, 2026, <https://blog.mozilla.ai/3w-for-in-browser-ai-webllm-wasm-webworkers/>
12. 7 Advanced JavaScript + Canvas Techniques for Data Visualization - Medium, accessed on May 8, 2026, <https://medium.com/uxdworld/7-advanced-javascript-canvas-techniques-for-data-visualization-e6236d72b754>
13. Procedural Techniques for Animation in Computer Graphics - TutorialsPoint, accessed on May 8, 2026, https://www.tutorialspoint.com/computer_graphics/computer_graphics_procedural_animation_techniques.htm
14. Creating Randomized Internet Traffic Animations with JavaScript Canvas - Medium, accessed on May 8, 2026, <https://medium.com/@python-javascript-php-html-css/creating-randomized-internet-traffic-animations-with-javascript-canvas-be6efcd87279>
15. markhliu/ml_animated: Use animation to explain machine learning - GitHub, accessed on May 8, 2026, https://github.com/markhliu/ml_animated
16. Procedural Animation with Canvas, random walks - DEV Community, accessed on May 8, 2026, <https://dev.to/benjaminolmsted/procedural-animation-with-canvas-random-walks-3joh>
17. Smoothly animate thousands of points with HTML5 Canvas and D3 - Bocoup,

- accessed on May 8, 2026,
<https://www.bocoup.com/blog/smoothly-animate-thousands-of-points-with-html5-canvas-and-d3>
18. Visualizing RAG: Building a Hands-On Tool to Understand Chunking, Embeddings, and Generation | by Jack Lakkapragada | Medium, accessed on May 8, 2026,
<https://medium.com/@lvjanakiram/visualizing-rag-building-a-hands-on-tool-to-understand-chunking-embeddings-and-generation-51621e4b78df>
 19. Validating machine learning models, accessed on May 8, 2026,
<https://data-science-gui.de/notebooks/41%20-%20Validating%20machine%20learning%20models.html>
 20. Validation – Introduction to Machine Learning in Python - The Carpentries Incubator, accessed on May 8, 2026,
<https://carpentries-incubator.github.io/machine-learning-novice-python/05-validation/index.html>
 21. 2D Grid-Based Game Prototype - Looking for Feedback on Mechanics and Fun Factor! : r/godot - Reddit, accessed on May 8, 2026,
https://www.reddit.com/r/godot/comments/1ivn0p5/2d_gridbased_game_prototype_looking_for_feedback/
 22. ML Experiment Tracking | MLflow AI Platform, accessed on May 8, 2026,
<https://mlflow.org/docs/latest/ml/tracking/>
 23. MLflow Tracking Quickstart, accessed on May 8, 2026,
<https://mlflow.org/docs/latest/ml/tracking/quickstart/>
 24. ML Experiment Tracking | MLflow AI Platform, accessed on May 8, 2026,
<https://mlflow.org/docs/latest/tracking.html#logging-to-a-local-directory>
 25. Reproducible Research Workflows: Docker and Conda for Simulation Projects - MatForge, accessed on May 8, 2026,
<https://matforge.org/reproducible-research-workflows-docker-and-conda-for-simulation-projects/>
 26. Getting Started with Python and FastAPI: A Complete Beginner's Guide - PyImageSearch, accessed on May 8, 2026,
<https://pyimagesearch.com/2025/03/17/getting-started-with-python-and-fastapi-a-complete-beginners-guide/>
 27. Bringing Python to Workers using Pyodide and WebAssembly - The Cloudflare Blog, accessed on May 8, 2026, <https://blog.cloudflare.com/python-workers/>
 28. Advanced animations - Canvas API - MDN Web Docs - Mozilla, accessed on May 8, 2026,
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Advanced_animations
 29. Making Very Small LLMs Smarter With RAG - Docker, accessed on May 8, 2026,
<https://www.docker.com/blog/making-small-llms-smarter/>
 30. RAG Tutorial: How to Build a RAG System on a Knowledge Graph - Neo4j, accessed on May 8, 2026, <https://neo4j.com/blog/developer/rag-tutorial/>
 31. I built a fully browser-native RAG and Semantic Search tool using WebGPU, Pyodide, and WASM. No servers, privacy-first. (MIT Licensed) - Reddit, accessed

on May 8, 2026,

https://www.reddit.com/r/opensource/comments/1qyqtzu/i_built_a_fully_browsernative_rag_and_semantic/

32. Visualizing RAG : r/LocalLLaMA - Reddit, accessed on May 8, 2026,
https://www.reddit.com/r/LocalLLaMA/comments/1q4jdeb/visualizing_rag/
33. Using FastAPI's TestClient for Quick API Validation - Shift Asia, accessed on May 8, 2026,
<https://shiftasia.com/community/using-fastapis-testclient-for-quick-api-validation/>
34. Testing - FastAPI, accessed on May 8, 2026,
<https://fastapi.tiangolo.com/tutorial/testing/>
35. Testing FastAPI Endpoints Without Spinning Up a Server - DEV Community, accessed on May 8, 2026,
https://dev.to/peytongreen_dev/testing-fastapi-endpoints-without-spinning-up-a-server-j74
36. Data Drift Monitoring and Its Importance in MLOps | by Sage Elliott | WhyLabs | Medium, accessed on May 8, 2026,
<https://medium.com/whylabs/understanding-data-drift-and-its-importance-in-mlops-69aac0bc1a07>
37. MLOps Series: Introduction to MLOps, Data Drift, Concept Drifts, and how to handle them in ML Production - Sameer Mahajan, accessed on May 8, 2026,
<https://mahajan-sameer.medium.com/mlops-series-introduction-to-mlops-data-drift-concept-drifts-and-how-to-handle-them-in-ml-e3821e05f948>
38. Population Stability Index (PSI) - machinelearningplus, accessed on May 8, 2026,
<https://machinelearningplus.com/deployment/population-stability-index-psi/>
39. population stability index (PSI) - Matthew Burke's Blog, accessed on May 8, 2026,
<https://mwburke.github.io/data%20science/2018/04/29/population-stability-index.html>
40. Data drift - DataRobot docs, accessed on May 8, 2026,
<https://docs.datarobot.com/en/docs/workbench/nxt-console/nxt-monitoring/nxt-data-drift.html>
41. Data Drift - What is Evidently? - Documentation, accessed on May 8, 2026,
https://docs.evidentlyai.com/metrics/preset_data_drift
42. Stealth Assessment: Meaning, Principles, Design And Model | Evelyn Learning, accessed on May 8, 2026,
<https://www.evelynlearning.com/blog/stealth-assessment-meaning-principles-design-and-model>